

PROYECTO: SERVICIOS TÉCNICOS

ETAPA 10 – SEGURIDAD, PRUEBAS FINALES, DOCUMENTACIÓN Y PUBLICACIÓN

Propósito

Realizar el cierre técnico del sistema Servicios Técnicos. El estudiante revisará prácticas básicas de seguridad, ejecutará pruebas finales, documentará la instalación y el uso del proyecto, organizará el repositorio Git y preparará una lista de requisitos para publicar la aplicación. El despliegue real depende de contar con servidor, credenciales y servicios disponibles.

1. Objetivo general

Entregar una versión funcional y documentada de Servicios Técnicos, con configuración protegida, pruebas registradas, repositorio actualizado y una guía clara para instalar, ejecutar y presentar el sistema.

2. Objetivos específicos

- Revisar autenticación, protección de rutas y validación de entradas.
- Evitar guardar contraseñas, claves y datos sensibles en el repositorio.
- Comprobar respuestas de la API y comportamiento ante errores.
- Ejecutar pruebas de regresión de Login, Clientes, Técnicos y Servicios.
- Crear un README con requisitos, instalación, configuración y ejecución.
- Ordenar el repositorio y guardar cambios en Git/GitHub.
- Preparar una lista de verificación para el despliegue sin publicar credenciales.

3. Requisitos previos

- Etapas 1 a 9 realizadas y aplicación integrada.
- MySQL con la base de datos tecnico.
- Backend Flask y frontend React funcionando localmente.
- Thunder Client disponible en VS Code.
- Repositorio Git inicializado y, si corresponde, conectado a GitHub.
- Permiso del docente para realizar pruebas y publicar el proyecto.



4. Actividad 1 – Respaldo y revisión del proyecto

1. Cierra las operaciones que estén modificando datos.
2. Realiza un respaldo de la base de datos antes de ejecutar pruebas que puedan eliminar o modificar registros.
3. Comprueba que los archivos principales estén en sus carpetas correspondientes.
4. Ejecuta git status y revisa qué archivos cambiaron.
5. No incluyas el entorno virtual, node_modules, archivos de contraseñas ni respaldos con información personal en el repositorio.

```
git status
git log --oneline -5
```

Si exportas la base de datos para una demostración, utiliza datos ficticios. No subas datos reales de clientes.

5. Actividad 2 – Revisión básica de seguridad

5.1 Credenciales y configuración

- Revisa que usuario y contraseña de MySQL no estén escritos directamente en archivos que se subirán a GitHub.
- Usa variables de entorno o un archivo local excluido por .gitignore para los secretos.
- Conserva un archivo .env.example con nombres de variables y valores de ejemplo no secretos.
- No compartas contraseñas, tokens ni claves en capturas, README o mensajes de error.

```
DB_HOST=localhost
DB_USER=usuario_de_ejemplo
DB_PASSWORD=CAMBIAR_ESTE_VALOR
DB_NAME=tecnico
FLASK_DEBUG=0
```

Guarda los valores reales en el archivo local .env, excluido del repositorio. Ejemplo de .gitignore:

```
.env
.env.*
!.env.example
venv/
.venv/
__pycache__/
*.py[cod]
frontend/node_modules/
frontend/dist/
```

Si una contraseña ya se subió a un repositorio, eliminarla del archivo no basta: debe cambiarse y revisarse el historial del repositorio.



5.2 Contraseñas de usuarios

- No guardar contraseñas de usuarios en texto plano.
- Si el login actual compara contraseñas directamente, registra esta corrección como prioritaria.
- En Flask, utiliza un método de hash como werkzeug.security.generate_password_hash y check_password_hash.
- No inventes ni restablezcas contraseñas de usuarios reales durante las pruebas.

```
from werkzeug.security import generate_password_hash, check_password_hash

# Al registrar o cambiar una contraseña:
hash_contrasena = generate_password_hash(contrasena)

# Al verificar el inicio de sesión:
if check_password_hash(hash_guardado, contrasena_ingresada):
    # credenciales válidas
    pass
```

Este ejemplo muestra el patrón; adapta el código a la estructura de tu tabla y al flujo real de autenticación.

5.3 Protección y validación

- Comprueba que las rutas internas de React estén protegidas.
- Proteger una ruta en React no protege por sí solo la API: los endpoints privados deben verificar autenticación en Flask.
- Valida en Flask los datos recibidos, aunque React ya los valide.
- Utiliza consultas parametrizadas para MySQL; no construyas SQL concatenando directamente datos del usuario.
- No muestres contraseñas, consultas SQL ni trazas internas completas al usuario.
- Desactiva el modo debug de Flask en un entorno publicado.
- En producción, usa HTTPS y configura CORS para permitir solo los orígenes necesarios.

6. Actividad 3 – Revisar consultas SQL

Usa consultas parametrizadas. Ejemplo conceptual con un cursor DB-API:

```
sql = "SELECT id, nombre FROM cliente WHERE id = %s"
cursor.execute(sql, (cliente_id,))
```

La sintaxis del marcador puede variar según el controlador MySQL instalado. Utiliza la sintaxis correspondiente a tu conector. Evita formar consultas concatenando valores.

Comprueba que UPDATE y DELETE utilicen el identificador esperado y que los errores se manejen de forma controlada.



7. Actividad 4 – Pruebas finales de regresión

Ejecuta las pruebas en un entorno de desarrollo con registros ficticios. Marca el resultado y escribe una observación cuando falle.

Módulo	Prueba / resultado esperado	Estado
Inicio de sesión	Credenciales válidas permiten entrar	☐
Inicio de sesión	Credenciales incorrectas no permiten entrar	☐
Cierre de sesión	Se elimina o invalida el estado de sesión	☐
Ruta protegida	Sin sesión no se accede a páginas privadas	☐
Clientes	GET muestra registros	☐
Clientes	POST crea un registro válido	☐
Clientes	PUT actualiza un registro	☐
Clientes	DELETE elimina un registro de prueba	☐
Técnicos	GET, POST, PUT y DELETE funcionan	☐
Servicios	GET muestra nombres de cliente y técnico	☐
Servicios	POST crea servicio con relaciones válidas	☐
Servicios	PUT actualiza descripción o estado	☐
Servicios	DELETE elimina un servicio de prueba	☐
Validación	Campos obligatorios vacíos son rechazados	☐
Error de API	La interfaz muestra un mensaje comprensible	☐
Conexión	La app informa el fallo si backend/DB no responde	☐

Registra cualquier fallo:

Prueba fallida / causa / corrección: _____

Prueba fallida / causa / corrección: _____

Prueba fallida / causa / corrección: _____

No consideres aprobada una prueba solo porque la pantalla se ve correcta: confirma también la respuesta de la API y el cambio real en la base de datos.

8. Actividad 5 – Pruebas con Thunder Client

- Prueba GET de Clientes, Técnicos y Servicios.
- Prueba POST con datos válidos.
- Prueba POST con campos faltantes y verifica una respuesta controlada.
- Prueba PUT con un ID existente y otro inexistente.
- Prueba DELETE con un registro de prueba.
- Registra método, endpoint, código HTTP y resultado observado.



- No incluyas contraseñas ni tokens reales en las evidencias.

Método y endpoint	Código recibido	Resultado	Observaciones

9. Actividad 6 – Crear el README.md

En la carpeta raíz del proyecto, crea README.md. Debe explicar qué hace el sistema y cómo ejecutarlo en otra computadora.

```
# Servicios Técnicos

## Descripción
Aplicación web para administrar clientes, técnicos y servicios técnicos.

## Tecnologías
- Frontend: React, JavaScript y CSS
- Backend: Python y Flask
- Base de datos: MySQL
- Pruebas de API: Thunder Client

## Requisitos
- Python y pip
- Node.js y npm
- MySQL
- Git

## Configuración
1. Crear la base de datos `tecnico`.
2. Ejecutar el script SQL del proyecto o crear las tablas requeridas.
3. Configurar las variables de entorno localmente.
4. Instalar dependencias del backend.
5. Instalar dependencias del frontend.

## Ejecución
Backend:
1. Activar el entorno virtual.
2. Ejecutar el punto de entrada de Flask.

Frontend:
1. Entrar a la carpeta frontend.
2. Ejecutar `npm run dev`.
```




```
## Módulos
- Login
- Dashboard
- Clientes
- Técnicos
- Servicios

## Pruebas
Describe los endpoints y los casos de prueba realizados.

## Autores
Escribe los nombres autorizados del equipo o estudiante.
```

Ajusta los comandos a los archivos reales. No escribas contraseñas ni publiques información personal.

10. Actividad 7 – Revisar dependencias y ejecución

Backend: verifica que exista un archivo de dependencias actualizado, por ejemplo requirements.txt. Con el entorno virtual activo, puedes generar una lista:

```
pip freeze > requirements.txt
```

Revisa el resultado y elimina dependencias que no correspondan al proyecto si fuera necesario. En frontend, confirma que package.json y package-lock.json estén actualizados. En una computadora limpia, las dependencias se instalan con npm install. No subas venv ni node_modules.

11. Actividad 8 – Git y GitHub

6. Revisa los archivos modificados.
7. Confirma que .env, contraseñas, archivos temporales y datos privados no estén incluidos.
8. Agrega README.md, .gitignore y los archivos del proyecto que sí deben versionarse.
9. Realiza un commit con un mensaje descriptivo.
10. Sube los cambios a la rama correspondiente.
11. Abre el repositorio remoto y confirma que los archivos y el README aparezcan correctamente.

```
git status
git add .
git status
git commit -m "Finaliza pruebas y documentacion del proyecto"
git push
```

Si detectas secretos antes del commit, quítalos del seguimiento y revisa .gitignore. No publiques el repositorio hasta resolverlo.



12. Actividad 9 – Preparación para publicación

- Elegir un hosting compatible con Flask y un servicio de MySQL.
- Configurar variables de entorno en el panel del hosting, no dentro del código.
- Configurar el frontend para usar la URL real de la API, no localhost.
- Configurar CORS para el dominio del frontend.
- Activar HTTPS.
- Desactivar debug y evitar mostrar errores internos.
- Crear una base de datos de producción con permisos limitados.
- Definir un plan de respaldo de la base de datos.
- Probar login, CRUD y relaciones en el entorno publicado.
- Confirmar con el docente antes de publicar datos o información institucional.

localhost identifica la computadora donde se ejecuta. Si el frontend se publica en Internet, las peticiones a localhost del navegador apuntarán al equipo del visitante, no al servidor de la aplicación.

13. Actividad 10 – Presentación final

12. Explica el propósito del sistema.
13. Muestra la arquitectura: React → API Flask → MySQL.
14. Inicia sesión y recorre el Dashboard.
15. Demuestra el CRUD de Clientes.
16. Demuestra el CRUD de Técnicos.
17. Registra un Servicio y muestra la relación con Cliente y Técnico.
18. Muestra una validación y un mensaje de error controlado.
19. Muestra una prueba de API en Thunder Client.
20. Explica dónde se configura la conexión a MySQL.
21. Muestra el README y el repositorio Git/GitHub.

14. Evidencias que se deben entregar

- Capturas del flujo completo de la aplicación.
- Captura de pruebas de Thunder Client con método, endpoint y código HTTP.
- Checklist de pruebas finales con resultados.
- README.md completo.
- Archivo .gitignore revisado.
- requirements.txt y package.json/package-lock.json actualizados según corresponda.
- Enlace al repositorio GitHub, si está autorizado.
- Lista de errores detectados y cómo se corrigieron.
- Captura de la aplicación funcionando sin errores críticos.



- Breve reflexión individual sobre lo aprendido.

15. Preguntas de reflexión

1. ¿Por qué no se deben subir credenciales a GitHub?

Respuesta: _____

_____

2. ¿Qué diferencia hay entre proteger una ruta de React y proteger un endpoint del backend?

Respuesta: _____

_____

3. ¿Por qué se deben validar datos tanto en frontend como en backend?

Respuesta: _____

_____

4. ¿Qué riesgo existe al almacenar contraseñas en texto plano?

Respuesta: _____

_____

5. ¿Qué función cumple README.md?

Respuesta: _____

_____

6. ¿Por qué localhost no debe utilizarse como URL de API en una aplicación publicada?

Respuesta: _____

_____

7. ¿Qué pruebas fueron más útiles para detectar errores y por qué?

Respuesta: _____

_____



8. ¿Qué mejoras implementarías en una siguiente versión?

Respuesta: _____

_____

16. Checklist final de entrega

- ☐ La aplicación inicia correctamente.
- ☐ Login y cierre de sesión fueron probados.
- ☐ Las rutas internas están protegidas.
- ☐ La API valida los datos recibidos.
- ☐ Las consultas SQL usan parámetros.
- ☐ Las contraseñas no se almacenan en texto plano.
- ☐ No hay secretos en el repositorio.
- ☐ Los CRUD de Clientes, Técnicos y Servicios fueron probados.
- ☐ Las relaciones Cliente/Técnico/Servicio funcionan.
- ☐ Los errores se muestran de forma controlada.
- ☐ README.md explica instalación y ejecución.
- ☐ .gitignore excluye secretos y carpetas generadas.
- ☐ Dependencias documentadas.
- ☐ Git commit y push realizados.
- ☐ Evidencias completas y sin datos sensibles.
- ☐ Se preparó la lista de pasos para despliegue.